

Project Report: Differential Evolution

Supervisors:

Dr. Trịnh Văn Chiến
MSc. Nguyễn Minh Quân

Participants:

Nguyễn Vũ Minh (202517032, ICT 02 – K70)
Ngô Công Nam (202517036, ICT 02 – K70)

Class: 166302 - IT3420E

Group: 20

June 14th, 2026

1 Overview

At its core, optimization is the process of finding the absolute best solution to a problem out of many possible choices. In the real world, this could mean finding the most aerodynamic shape for an airplane wing, or tweaking the settings of a complex machine to minimize energy consumption. Often, these problems are too complex to solve with standard mathematical formulas because they involve unpredictable, jagged, or multi-dimensional data.

Differential Evolution (DE) is a powerful, computer-based method designed to solve these exact types of difficult optimization problems. Instead of guessing randomly or trying every single possibility one by one—which would take a computer centuries—DE uses a smart, population-based strategy. It starts by creating a “population” of multiple random candidate solutions. It then improves these solutions over and over through a simple three-step cycle:

- **Mutation:** It combines existing solutions to create new, slightly altered variations.
- **Crossover:** It mixes traits from the successful variations with the original solutions.
- **Selection:** It compares the new creations against the old ones in a survival-of-the-fittest matchup, keeping only the superior performer.

By repeating this cycle, the entire group of solutions naturally drifts toward the best possible outcome. DE is highly favored in modern computing because it requires very little tuning, handles chaotic data effortlessly, and is incredibly efficient at finding global solutions where other methods get stuck.

2 Core Problem

In engineering and mathematics, a problem is typically modeled as a **cost function** (or objective function). This function takes a set of input variables (parameters) and calculates a single value representing the “cost” of that specific combination. Our goal is **Global Optimization**: finding a set of parameters that produces the absolute lowest possible value (the global minimum) across

the entire search space. That is, finding a vector x^* such that:

$$x^* = \arg \min_{x \in \Omega} f(x) \quad (1)$$

where:

- $f(x)$ is the objective function to minimize.
- x is a vector of D real-valued parameters: $x = [x_1, x_2, \dots, x_D]$.
- Ω is the feasible search space bounded by lower and upper limits:

$$X_j^{(\min)} \leq x_j \leq X_j^{(\max)} \quad \text{for all } j = 1, 2, \dots, D. \quad (2)$$

Standard calculus-based methods rely on derivatives (gradients) to find the slope of a curve and slide down to the bottom. However, real-world problems are frequently **non-differentiable, discontinuous, and multi-modal** (containing many false bottoms or “local minima”). If an algorithm gets trapped in a local minimum, it fails to find the true optimal solution. DE is specifically designed to navigate these rugged landscapes without getting stuck.

3 Algorithm Design

Differential Evolution optimizes the target function by maintaining a population of N candidate solution vectors. Each vector represents a point in the D -dimensional search space. The algorithm is **iterative**, meaning it processes the entire population repeatedly through a sequence of cycles. Each complete cycle of processing every candidate solution exactly once is defined as a **generation**. At generation g , the current state of the population can be denoted as:

$$P_g = \{x_{i,g} \mid i = 1, 2, \dots, N\} \quad (3)$$

The algorithm transitions from the current generation (g) to the next generation ($g + 1$) by executing four primary phases: Initialization, Mutation, Crossover, and Selection.

3.1 Initialization

This step happens only once at the very beginning (generation $g = 0$). The algorithm randomly scatters the population across the search space within the specified boundaries. For each i -th vector and its j -th parameter:

$$x_{i,j,0} = X_j^{(\min)} + \text{rand}(0, 1) \times (X_j^{(\max)} - X_j^{(\min)}) \quad (4)$$

For clarity, candidate solution vectors are denoted as $x_{i,g}$, representing the i -th vector in the population at generation g . Individual components within a vector are denoted as $x_{i,j,g}$, representing the specific value of the j -th parameter of that vector.

3.2 Mutation

For each target vector in the current generation, DE selects three other distinct vectors $x_{r_1,g}$, $x_{r_2,g}$, $x_{r_3,g}$ completely at random from the same generation. Then it computes the spatial difference between two of them, scales it by a mutation factor F , and adds it to the third (the base vector). This creates a **mutated donor vector** $v_{i,g}$:

$$v_{i,g} = x_{r_1,g} + F \times (x_{r_2,g} - x_{r_3,g}) \quad (5)$$

where $F \in [0, 2]$ is a user-defined **scaling factor** that controls the step size of the search.

The primary purpose of the donor vector is to automatically guide the direction and step size of the search based on the current spread of the population. Early in the process, when solutions are far apart, the difference vector $x_{r_2,g} - x_{r_3,g}$ is large, allowing the donor vector to perform wide exploration across the search space. As the population converges around promising areas, this difference naturally shrinks, automatically shifting the donor vector's role toward fine-tuning and precise local exploitation without requiring manual adjustment.

3.3 Crossover

To increase diversity, DE mixes parameters of the original target vector $x_{i,g}$ with the mutated donor vector $v_{i,g}$ to form a **trial vector** $u_{i,g}$. For each parameter j , a random number between 0 and 1 is generated. If it is less than or equal to a user-defined Crossover Probability ($CR \in [0, 1]$), the parameter is taken from the donor vector; otherwise, it is kept from the target vector:

$$u_{i,j,g} = \begin{cases} v_{i,j,g} & \text{if } \text{rand}(0, 1) \leq CR \text{ or } j = j_{\text{rand}} \\ x_{i,j,g} & \text{otherwise} \end{cases} \quad (6)$$

where j_{rand} is a random index being chosen in the range $[1, D]$ for each vector to ensure that the trial vector inherits at least one parameter from the donor vector.

The crossover step acts as a balancing mechanism between exploration and stability. If the algorithm were to accept the mutated donor vector entirely, the multi-dimensional leap might be too disruptive, destroying the good characteristics already found in the target vector. Conversely, if no parameters were changed, the generation would stagnate, resulting in wasted computational effort. By mixing components of both vectors, crossover regulates this step size, allowing the algorithm to test novel combinations while retaining a reliable baseline.

3.4 Selection

Finally, the algorithm determines whether the newly generated trial vector $u_{i,g}$ is actually better than the original target vector $x_{i,g}$. This is a deterministic, one-on-one survival mechanism based strictly on the cost function value $f(x)$:

$$x_{i,g+1} = \begin{cases} u_{i,g} & \text{if } f(u_{i,g}) \leq f(x_{i,g}) \\ x_{i,g} & \text{otherwise} \end{cases} \quad (7)$$

If the trial vector yields an equal or lower cost, it replaces the target vector and advances to the next generation ($g + 1$). Otherwise, the target vector survives intact into the next generation.

This loop of Mutation, Crossover, and Selection repeats generation after generation ($g = 0, 1, 2, \dots$) until a termination criterion—such as a maximum number of generations or a target accuracy—is met.

4 Real-world Applications

In modern engineering, systems are becoming increasingly intricate. This complexity requires optimization methods that can handle high-dimensional, noisy, and non-linear design spaces where traditional gradient-based calculus fails. Differential Evolution has become a cornerstone tool in these domains due to its structural robustness and computational efficiency.

Within the field of electronics, DE is heavily utilized for automated circuit design and component optimization. For instance, it is used to precisely tune the parameters—such as resistance, capacitance, and transistor geometries—of analog amplifiers to maximize voltage gain and frequency response while minimizing power consumption.

Additionally, DE plays a vital role in signal processing and electromagnetics. Engineers leverage the algorithm to design optimal digital filters with minimal phase distortion, as well as to synthesize the geometric configurations of high-frequency antenna arrays to maximize directional signal gain and reduce interference.

Beyond electronics, DE serves a crucial role across broader engineering disciplines. In control systems, it is used to automatically tune complex controller coefficients for industrial robotics. In power engineering, it optimizes economic load dispatching across electrical grids, and in aerospace engineering, it refines the structural components and aerodynamic profiles of aircraft surfaces.

5 Example Optimization

Minimize the cost function:

$$f(x_0, x_1) = 0.2x_0^2 + 0.1x_1^2 - \cos(2\pi x_0) \cos(2\pi x_1) + e^{\frac{\sin(x_0+x_1)}{4}} \quad (8)$$

subject to the following constraints:

$$-1.5 \leq x_0 \leq 4.5, \quad -4.5 \leq x_1 \leq 1.5 \quad (9)$$

Uniformly distributing 50 points into the xy -plane and executing the DE algorithm yields a progression where the population clusters and converges dynamically from generations $g = 0$ to the final generation $g = 84$.

The optimal solution reached is:

$$f(x^*) \approx -0.1148 \quad \text{at } (x_0, x_1) \approx (-0.4977, -0.5002) \quad (10)$$

(Visualizations, C++ source code and benchmark logs can be found in the archive.)

6 Advanced Variant: SHADE

While the original Differential Evolution algorithm is highly effective, its performance depends heavily on the user manually tuning the mutation scaling factor (F) and the crossover rate (CR). If these parameters are set incorrectly, the algorithm can easily stall or overshoot the global minimum.

To solve this, researchers developed **SHADE (Success-History Based Differential Evolution)**, which is currently one of the most powerful and widely used adaptations of the original DE idea. SHADE introduces major structural improvements over classic DE by automating parameter selection and modifying how mutant vectors are directed:

6.1 Self-adaptive parameters F and CR

Instead of forcing the user to guess a fixed, global value for F and CR , SHADE maintains a **historical memory** of parameter pairs that successfully produced superior offspring in recent generations.

- **Mechanism:** For every individual vector in the population, unique F and CR values are sampled from a probability distribution centered around a randomly selected successful pair from the history log.
- **Improvement:** If a generated vector successfully defeats its parent, the F and CR values that created it are saved into the history layout, dynamically driving the population toward the most effective step sizes for that specific stage of the search.

6.2 The “current-to-best” mutation strategy

Original DE uses a completely random base vector (x_{r_1}), which can make the search wander aimlessly in later generations. SHADE replaces this with an elite-guided mutation framework:

$$v_{i,g} = x_{i,g} + F \times (x_{\text{pbest},g} - x_{i,g}) + F \times (x_{r_2,g} - x_{r_3,g}) \quad (11)$$

- **Mechanism:** The mutant donor vector balances its trajectory by utilizing x_{pbest} , which is a vector randomly chosen from the top $p\%$ of the current population (usually the top 10% to 20% best-performing vectors).
- **Improvement:** This shifts the mutation from a completely blind exploration to a greedier, target-driven exploration. It dramatically accelerates convergence speed because vectors are actively pulled toward high-performance zones without entirely sacrificing population diversity.

6.3 External archive with queue-based eviction

To prevent the elite-guided mutation from causing premature convergence, SHADE introduces a fixed-size external archive that functions logically as a First-In, First-Out (FIFO) queue.

- **Mechanism:** When a parent vector is defeated during the Selection phase, it is pushed to the tail of the archive queue. Once the archive reaches its capacity limit (typically equal to the population size N), any incoming defeated parent automatically evicts the oldest vector at the head of the queue. During mutation, the indexes r_2, r_3 can be randomly chosen from either the active population or this queue.
- **Improvement:** Using a strictly limited queue provides two vital benefits. First, it preserves structural history; if the active population begins clustering too tightly, the queue offers alternative coordinates to maintain healthy genetic diversity. Second, limiting the queue length ensures that obsolete, high-cost data from early generations is discarded. This prevents ancient, unoptimized coordinates from creating massive, chaotic difference vectors that would otherwise disrupt and ruin the fine-tuning of newly optimized clusters.

7 Related Algorithms

Within the broader paradigm of Evolutionary Computation, Differential Evolution (DE) shares its closest theoretical roots with **Genetic Algorithms (GA)** and **Evolution Strategies (ES)**. All three algorithms are inspired by the principles of biological evolution and natural selection, utilizing a population of candidate solutions that iteratively adapt to find the global optimum.

7.1 Genetic Algorithms (GA)

Originally designed by John Holland, Genetic Algorithms were engineered primarily to solve discrete and combinatorial optimization problems. GA represents solutions as chromosome-like strings—traditionally using binary arrays of 0s and 1s. It selects pairs of high-performing individuals to undergo crossover (recombination) by swapping chunks of their strings, followed by a mutation step that randomly flips bits with a low probability.

DE was explicitly developed to adapt the core philosophy of GA for continuous, real-valued numerical spaces. Instead of swapping discrete binary bits, DE alters values by calculating continuous directional vectors across a floating-point landscape.

7.2 Evolution Strategies (ES)

Developed concurrently by Ingo Rechenberg and Hans-Paul Schwefel, Evolution Strategies were designed from the ground up for continuous parameter optimization, making them a direct functional sibling to DE. Rather than relying heavily on crossover, classic ES relies predominantly on mutation as its primary search engine. It mutates real-valued vectors by adding random noise sampled from a Gaussian (normal) distribution, dynamically adjusting the step size (variance) as the search progresses.

While both target continuous spaces, DE improves upon the ES mutation concept by eliminating the need for rigid Gaussian probability distributions. Instead, DE dynamically derives its mutation step size directly from the spatial distance between its own population members ($x_{r_2} - x_{r_3}$), allowing the algorithm to automatically scale its steps based on current population diversity.

7.3 Architectural Similarities

Despite differences in data representation and mutation mechanics, GA, ES, and DE are bound by a shared operational pipeline as they all map directly to the same four-stage evolutionary lifecycle:

- **Population-based initialization:** All three algorithms begin by scattering a parallel pool of candidate solutions across the search bounds to maximize initial coverage.
- **Stochastic variation (Mutation & Crossover):** They alter genetic material to explore the landscape. While GA crosses over before mutating, and DE mutates before crossing over, both look to blend existing traits while introducing trial variations.
- **Fitness-driven selection:** They use a deterministic selection wrapper to enforce survival of the fittest. Whether evaluating a tournament pool in GA, an elite population strategy in ES, or direct one-to-one parent-child duels in DE, individual trajectory is always gated by the objective function's cost evaluation.